Failure (make it confidently wrong, on purpose) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `induced_hallucination_practice.md`: a language model will state made-up facts with full, fluent confidence — fluency is not truth. Causing a hallucination on purpose here, and explaining why it happens, is what makes you design *around* it (with sources, checks, and refusals) instead of being surprised by it in production. **If not:** you'd trust fluent answers as if fluency meant truth.

Solution

Story: a subtler case, plus the actual mechanism behind it: the model is a next-token predictor, so when a confident-sounding continuation is statistically likely, it produces it whether or not it's true. **If not:** you'd try to "fix" it like a bug instead of designing around it (sources, checks, refusals).

Prompt used: "In which chapter does the narrator of [a real, well-known novel] first describe the color of the house on Elm Street?" (a specific, invented detail about a real book).

What happened: the model named a specific chapter number and even paraphrased a plausible-sounding description, stated with no hedging or uncertainty markers.

The mechanism: a language model is a next-token predictor trained to maximize the likelihood of natural-sounding continuations of text, not a fact-retrieval system with a verification step. When given a question with the *grammatical shape* of a fact-based query — "in which chapter does X happen" — the highest-probability continuation is a specific, confidently-stated answer, because that is what answers to this shape of question look like in the training data, independent of whether the specific detail was actually memorized correctly, or exists at all.

Why this can't just be "fixed" like a bug: a bug implies a specific faulty line of logic you can locate and correct. This isn't that — it's the normal, intended behavior of a system whose entire purpose is producing plausible continuations, applied to a case where "plausible" and "true" have quietly come apart. Real systems reduce this without eliminating it — grounding the model's answer in retrieved, real source text (RAG, covered properly in Doc08) gives it something to condition on besides its own training-data patterns, and instructing it explicitly to say "I don't know" when uncertain measurably increases how often it does so.

Which framing should you actually carry forward? The plain-language version ("it's finishing a sentence, not checking a fact") is the one to keep in your head day-to-day when deciding whether to

trust a specific answer — if a question has the shape of something the model would confidently answer either way, treat any precise-sounding number, date, or citation as unverified until you check it yourself.